

Notes for the Project

[X] math (Tensors (vectors/matrices))

[X] find gradients of arbitrary functions

[] Glue

$$f: x \longrightarrow y \quad (\text{word in, word out})$$

in this project we'll do MNIST:

cost function = how bad a model is

you take in the training data and output and compare them, and then change the weights based on that.

Goal: minimize cost function

we coded all the various matrix functions.

Now we need to implement autodifferentiation

- **Numerical Differentiation**

$$\frac{f(x+h) - f(x)}{h}$$

this is fine but we have thousands of parameters, so inefficient.

- **Symbolic Differentiation**

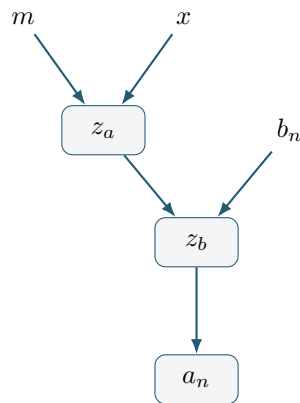
u give an input string and it then produces the symbols that gives u the required output.

ridiculously hard to implement. (im ngl i do NOT know what that means)

according to chatGPT: Symbolic differentiation represents a mathematical expression as a tree or graph and applies rules such as the product and chain rules to construct a new, exact derivative expression—for example, transforming $y = x^2 + 3x$ into $y' = 2x + 3$. Autograd instead records numerical operations in a computational graph and propagates derivative values backward at the current inputs, producing gradients rather than a simplified symbolic formula; your C library implements this reverse-mode autograd approach.

- autograd

given $f(x) = mx + b$. we define the computation graph for the function



perceptron:

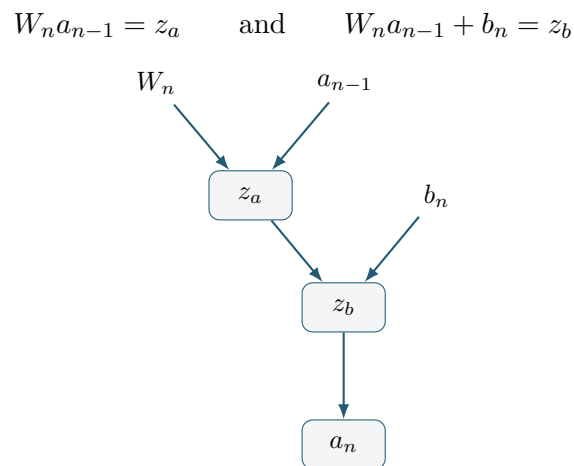


doesn't allow for residual connection (so u can't jump from a to c).

In a FF NN, output of some layer N is

$$\text{ReLU}(W_n a_{n-1} + b_n)$$

let



we need to topological sort. that graph is directed acyclic graph. its directed coz nodes are some order to them and its acyclic becasue u can't loop through it. two arbitrary elements are not related here.

topological sorting = resolving all dependencies before moving onto the next node

so a logical topological sort for the graph above (looking forward) would be

$$\underbrace{W_n \longrightarrow a_{n-1}}_1 \longrightarrow \underbrace{z_a \longrightarrow b_n}_2 \longrightarrow \underbrace{z_b \longrightarrow a_n}_3$$

notice you cannot go down a level without first resolving all the dependencies on that level first.

this is implemented via DFS.

anytime u get to a node, u pop it off the stack and add it to the visited, and then push it back on the stack and then push its children on the stack.

now when u start going down the stack, we see we have already visited the node, we can add it to the output.

need to find the partial derivatives of the cost w.r.t to each of the arguments

$$a_n = \text{ReLU}(\text{add}(\text{mul}(W_n, a_{n-1}), b_n))$$

$$1. a_n = \text{ReLU}(z_b)$$

$$2. z_b = \text{add}(z_a, z_b)$$

$$3. z_a = \text{mul}(W_n, a_{n-1})$$

-partial derivative of z_a w.r.t $W_n = a_{n-1}$ (product rule of derivatives)

this is saying that z_a changes a_{n-1} w.r.t W_n .

-partial der. of z_b w.r.t $z_a = 1$ (again js differentiation and since b_n is being added it goes 1)

this is saying that z_b changes by a constant (1) w.r.t W_n .

-partial der. of a_n w.r.t z_b as the derivative of ReLU.

now the interesting bit in backpropagation: to find the partial derivative of the final node w.r.t z_b (so when u finally reach the end of the chain) its just product rule (so product of all the pds before the final node and the very first input in our case being W_n to a_n)

$$\text{pd}(\text{final} \rightarrow \text{first}) = \text{pd}(2) \cdot \text{pd}(3) \cdot \text{pd}(1)$$

$$f(a, b) = a + b$$

$$\frac{\partial f}{\partial a} = 1 \quad \frac{\partial f}{\partial b} = 1$$

to see how this relates to the cost function,

$$\frac{\partial C}{\partial \text{pdf}(a)} = \frac{\partial f}{\partial a} \cdot \frac{\partial c}{\partial f} \quad \{\text{via the chain rule}\}$$

Cross entropy

$$\text{Cross entropy}(p, q) = -p \ln(q)$$

$$\frac{\partial \text{CE}}{\partial p} = -\ln q \quad \frac{\partial \text{CE}}{\partial q} = -\frac{p}{q}$$

$$\text{softmax}_i = \frac{e^{a_i}}{\sum_j e^{a_j}}$$

ok so we have now implemented `model_prog_compute` and `model_prog_compute_grads` (lets call `mpc` and `mpcg`)

`mpc` does a topological sort of the directed acyclical graph and gives you a list of the values in the order its meant to be traversed through so you can then do chain rule on it easier

`mpcg` traverses in reverse topological order and computed the gradient of each value w.r.t to the final output

You'll often see the words arena and custom `PUSH_ARENA` functions here. This is an implementation of a custom memory allocator function.

essentially, rather than having to allocate memory dynamically for each and every variable (which in ML is often arrays and matrices), you can do something called a large buffer, where u basically beg the OS for a big memory chunk and then does all the memory allocation within it.

think about like this: view `malloc` as asking the front desk for a hotel room in the Grand Budapest Hotel. Always busy and poor Zero has to run all over the place to find a big enough room for you. Whereas, a custom `malloc` (large buffer) aka arena alloc, basically pre-books however many rooms you need.

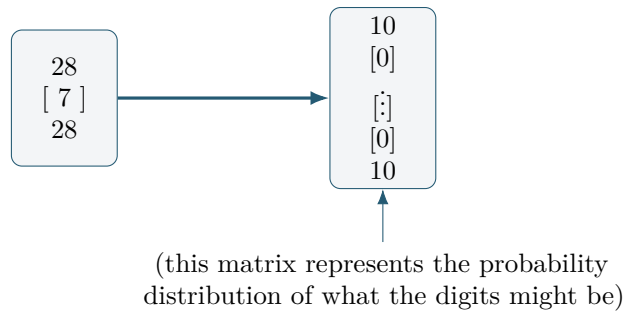
This is also better for performance as the memory itself is interacting with other parts in contiguous blocks and can assign memory off the stack in $O(1)$ time. so instead of search, its just placing the memory in a spot.

As a future expansion, I want to implement it.

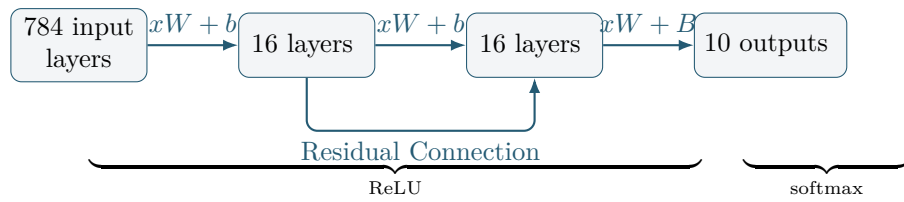
some blogs for ur reference and mine:

1. <https://www.gingerbill.org/article/2019/02/08/memory-allocation-strategies-002/>
 2. <https://nullprogram.com/blog/2023/09/27/>
-

MNIST is a 28×28 pixel handwritten digit



The SOTA for CV is CNNs, but for the sake of simplicity, we will do a simple MLP (a multi layer perceptron)



loss = cross entropy.